

统一软件开发过程(RUP)的概念和方法

统一软件开发过程 (Rational Unified Process,RUP)是一种面向对象且基于网络的程序开发方法论。

根据 Rational(Rational Rose 和统一建模语言的开发者)的说法, 好像一个在线的指导者, 它可以为所有方面和层次的程序开发提供指导方针, 模版以及事例支持。RUP 和类似的产品--例如面向对象的软件过程 (OOSP), 以及 OPEN Process 都是理解性的软件工程工具--把开发中面向过程的方面(例如定义的阶段, 技术和实践)和其他开发的组件(例如文档, 模型, 手册以及代码等等)整合在一个统一的框架内。

一.六大经验

迭代式开发。在软件开发的早期阶段就想完全、准确的捕获用户的需求几乎是不可能的。实际上, 我们经常遇到的问题是需求在整个软件开发工程中经常会改变。迭代式开发允许在每次迭代过程中需求可能有变化, 通过不断细化来加深对问题的理解。迭代式开发不仅可以降低项目的风险, 而且每个迭代过程以可以执行版本结束, 可以鼓舞开发人员。

管理需求。确定系统的需求是一个连续的过程, 开发人员在开发系统之前不可能完全详细的说明一个系统的真正需求。RUP 描述了如何提取、组织系统的功能和约束条件并将其文档化, 用例和脚本的使用以被证明是捕获功能性需求的有效方法。

基于组件的体系结构。组件使重用成为可能, 系统可以由组件组成。基于独立的、可替换的、模块化组件的体系结构有助于管理复杂性, 提高重用率。RUP 描述了如何设计一个有弹性的、能适应变化的、易于理解的、有助于重用的软件体系结构。

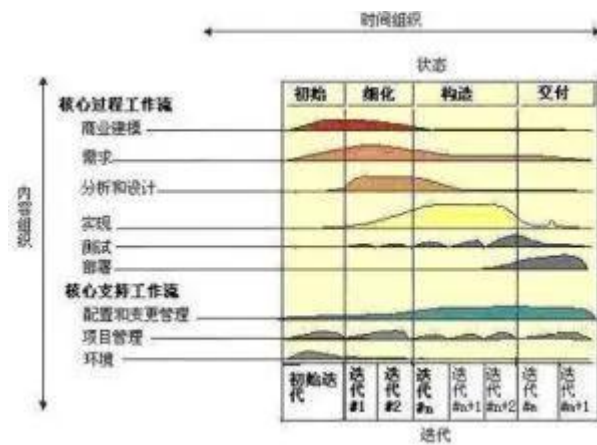
可视化建模。RUP 往往和 UML 联系在一起，对软件系统建立可视化模型帮助人们提供管理软件复杂性的能力。RUP 告诉我们如何可视化的对软件系统建模，获取有关体系结构于组件的结构和行为信息。

验证软件质量。在 RUP 中软件质量评估不再是事后进行或单独小组进行的分离活动，而是内建于过程中的所有活动，这样可以及早发现软件中的缺陷。

控制软件变更。迭代式开发中如果没有严格的控制和协调，整个软件开发过程很快就陷入混乱之中，RUP 描述了如何控制、跟踪、监控、修改以??品，隔离来自其他工作空间的变更，以此为每个开发人员建立安全的工作空间。

二.统一软件开发过程 RUP 的二维开发模型

RUP 软件开发生命周期是一个二维的软件开发模型。横轴通过时间组织，是过程展开的生命周期特征，体现开发过程的动态结构，用来描述它的术语主要包括周期(Cycle)、阶段(Phase)、迭代(Iteration)和里程碑(Milestone)；纵轴以内容来组织为自然的逻辑活动，体现开发过程的静态结构，用来描述它的术语主要包括活动(Activity)、产物(Artifact)、工作者(Worker)和工作流(Workflow)。如图 1：



三.统一软件开发过程 RUP 核心概念

RUP 中定义了一些核心概念，如下图：



角色：描述某个人或者一个小组的行为与职责。RUP 预先定义了很多角色。

活动：是一个有明确目的的独立工作单元。

工件：是活动生成、创建或修改的一段信息。

四.统一软件开发过程 RUP 裁剪

RUP 是一个通用的过程模板，包含了很多开发指南、制品、开发过程所涉及到的角色说明，由于它非常庞大所以对具体的开发机构和项目，用 RUP 时还要做裁剪，也就是要对 RUP 进行配置。RUP 就像一个元过程，通过对 RUP 进行裁剪可以得到很多不同的开发过程，这些软件开发过程可以看作 RUP 的具体实例。RUP 裁剪可以分为以下几步：

- 1) 确定本项目需要哪些工作流。RUP 的 9 个核心工作流并不总是需要的，可以取舍。
- 2) 确定每个工作流需要哪些制品。
- 3) 确定 4 个阶段之间如何演进。确定阶段间演进要以风险控制为原则，决定每个阶段要那些工作流，每个工作流执行到什么程度，制品有那些，每个制品完成到什么程度。
- 4) 确定每个阶段内的迭代计划。规划 RUP 的 4 个阶段中每次迭代开发的内容。
- 5) 规划工作流内部结构。工作流涉及角色、活动及制品，他的复杂程度与项目规模即角色多少有关。最后规划工作流的内部结构，通常用活动图的形式给出。

五.开发过程中的各个阶段和里程碑

RUP 中的软件生命周期在时间上被分解为四个顺序的阶段，分别是：初始阶段(Inception)、细化阶段(Elaboration)、构造阶段(Construction)和交付阶段(Transition)。每个阶段结束

于一个主要的里程碑(Major Milestones); 每个阶段本质上是两个里程碑之间的时间跨度。在每个阶段的结尾执行一次评估以确定这个阶段的目标是否已经满足。如果评估结果令人满意的话, 可以允许项目进入下一个阶段。

1. 初始阶段

初始阶段的目标是为系统建立商业案例并确定项目的边界。为了达到该目的必须识别所有与系统交互的外部实体, 在较高层次上定义交互的特性。本阶段具有非常重要的意义, 在这个阶段中所关注的是整个项目进行中的业务和需求方面的主要风险。对于建立在原有系统基础上的开发项目来讲, 初始阶段可能很短。 初始阶段结束时是第一个重要的里程碑: 生命周期目标(Lifecycle Objective)里程碑。生命周期目标里程碑评价项目基本的生存能力。

2. 细化阶段

细化阶段的目标是分析问题领域, 建立健全的体系结构基础, 编制项目计划, 淘汰项目中最高风险的元素。为了达到该目的, 必须在理解整个系统的基础上, 对体系结构作出决策, 包括其范围、主要功能和诸如性能等非功能需求。同时为项目建立支持环境, 包括创建开发案例, 创建模板、准则并准备工具。 细化阶段结束时第二个重要的里程碑: 生命周期结构(Lifecycle Architecture)里程碑。生命周期结构里程碑为系统的结构建立了管理基准并使项目小组能够在构建阶段中进行衡量。此刻, 要检验详细的系统目标和范围、结构的选择以及主要风险的解决方案。

3. 构造阶段

在构建阶段,所有剩余的构件和应用程序功能被开发并集成为产品,所有的功能被详细测试。

从某种意义上说,构建阶段是一个制造过程,其重点放在管理资源及控制运作以优化成本、进度和质量。构建阶段结束时是第三个重要的里程碑:初始功能(Initial Operational)里程碑。初始功能里程碑决定了产品是否可以在测试环境中进行部署。此刻,要确定软件、环境、用户是否可以开始系统的运作。此时的产品版本也常被称为“beta”版。

4. 交付阶段

交付阶段的重点是确保软件对最终用户是可用的。交付阶段可以跨越几次迭代,包括为发布做准备的产品测试,基于用户反馈的少量的调整。在生命周期的这一点上,用户反馈应主要集中在产品调整,设置、安装和可用性问题,所有主要的结构问题应该已经在项目生命周期的早期阶段解决了。在交付阶段的终点是第四个里程碑:产品发布(Product Release)里程碑。此时,要确定目标是否实现,是否应该开始另一个开发周期。在一些情况下这个里程碑可能与下一个周期的初始阶段的结束重合。

六.统一软件开发过程 RUP 的核心工作流(Core Workflows)

RUP 中有 9 个核心工作流,分为 6 个核心过程工作流(Core Process Workflows)和 3 个核心支持工作流(Core Supporting Workflows)。尽管 6 个核心过程工作流可能使人想起传统瀑布模型中的几个阶段,但应注意迭代过程中的阶段是完全不同的,这些工作流在整个生命周期中一次又一次被访问。9 个核心工作流在项目中轮流被使用,在每一次迭代中以不同的重点和强度重复。

1. 商业建模(Business Modeling)

商业建模工作流描述了如何为新的目标组织开发一个构想,并基于这个构想在商业用例模型和商业对象模型中定义组织的过程,角色和责任。

2. 需求(Requirements)

需求工作流的目标是描述系统应该做什么,并使开发人员和用户就这一描述达成共识。为了达到该目标,要对需要的功能和约束进行提取、组织、文档化;最重要的是理解系统所解决问题的定义和范围。

3. 分析和设计(Analysis & Design)

分析和设计工作流将需求转化成未来系统的设计,为系统开发一个健壮的结构并调整设计使其与实现环境相匹配,优化其性能。分析设计的结果是一个设计模型和一个可选的分析模型。设计模型是源代码的抽象,由设计类和一些描述组成。设计类被组织成具有良好接口的设计包(Package)和设计子系统(Subsystem),而描述则体现了类的对象如何协同工作实现用例的功能。设计活动以体系结构设计为中心,体系结构由若干结构视图来表达,结构视图是整个设计的抽象和简化,该视图中省略了一些细节,使重要的特点体现得更加清晰。体系结构不仅仅是良好设计模型的承载媒介,而且在系统的开发中能提高被创建模型的质量。

4. 实现(Implementation)

实现工作流的目的包括以层次化的子系统形式定义代码的组织结构;以组件的形式(源文件、二进制文件、可执行文件)实现类和对象;将开发出的组件作为单元进行测试以及集成由单个开发者(或小组)所产生的结果,使其成为可执行的系统。

5. 测试(Test)

测试 workflow 要验证对象间的交互作用, 验证软件中所有组件的正确集成, 检验所有的需求已被正确的实现, 识别并确认缺陷在软件部署之前被提出并处理。RUP 提出了迭代的方法, 意味着在整个项目中进行测试, 从而尽可能早地发现缺陷, 从根本上降低了修改缺陷的成本。测试类似于三维模型, 分别从可靠性、功能性和系统性能来进行。

6. 部署(Deployment)

部署 workflow 的目的是成功的生成版本并将软件分发给最终用户。部署 workflow 描述了那些与确保软件产品对最终用户具有可用性相关的活动, 包括: 软件打包、生成软件本身以外的产品、安装软件、为用户提供帮助。在有些情况下, 还可能包括计划和进行 beta 测试版、移植现有的软件和数据以及正式验收。

7. 配置和变更管理(Configuration & Change Management)

配置和变更管理工作 flow 描绘了如何在多个成员组成的项目中控制大量的产物。配置和变更管理工作 flow 提供了准则来管理演化系统中的多个变体, 跟踪软件创建过程中的版本。工作 flow 描述了如何管理并行开发、分布式开发、如何自动化创建工程。同时也阐述了对产品修改原因、时间、人员保持审计记录。

8. 项目管理(Project Management)

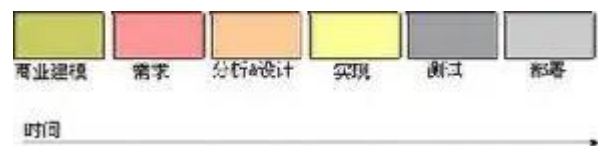
软件项目管理平衡各种可能产生冲突的目标, 管理风险, 克服各种约束并成功交付使用户满意的产品。其目标包括: 为项目的管理提供框架, 为计划、人员配备、执行和监控项目提供实用的准则, 为管理风险提供框架等。

9. 环境(Environment)

环境工作流的目的是向软件开发组织提供软件开发环境，包括过程和工具。环境工作流集中于配置项目过程中所需要的活动，同样也支持开发项目规范的活动，提供了逐步的指导手册并介绍了如何在组织中实现过程。

七、RUP 的迭代开发模式

RUP 中的每个阶段可以进一步分解为迭代。一个迭代是一个完整的开发循环，产生一个可执行的产品版本，是最终产品的一个子集，它增量式地发展，从一个迭代过程到另一个迭代过程到成为最终的系统。传统上的项目组织是顺序通过每个工作流，每个工作流只有一次，也就是我们熟悉的瀑布生命周期（见图 2）。这样做的结果是到实现末期产品完成并开始测试，在分析、设计和实现阶段所遗留的隐藏问题会大量出现，项目可能要停止并开始一个漫长的错误修正周期。



一种更灵活，风险更小的方法是多次通过不同的开发工作流，这样可以更好的理解需求，构造一个健壮的体系结构，并最终交付一系列逐步完成的版本。这叫做一个迭代生命周期。在工作流中的每一次顺序的通过称为一次迭代。软件生命周期是迭代的连续，通过它，软件是增量的开发。一次迭代包括了生成一个可执行版本的开发活动，还有使用这个版本所必需的其他辅助成分，如版本描述、用户文档等。因此一个开发迭代在某种意义上是在所有工作流

中的一次完整的经过，这些工作流至少包括：需求工作流、分析和设计工作流、实现工作流、测试工作流。其本身就像一个小瀑布项目（见图 3）。



图 3 RUP 的迭代模型

与传统的瀑布模型相比较，迭代过程具有以下优点：

降低了在一个增量上的开支风险。如果开发人员重复某个迭代，那么损失只是这一个开发有误的迭代的花费。

降低了产品无法按照既定进度进入市场的风险。通过在开发早期就确定风险，可以尽早来解决而不至于在开发后期匆匆忙忙。

加快了整个开发工作的进度。因为开发人员清楚问题的焦点所在，他们的工作会更有效率。

由于用户的需求并不能在一开始就作出完全的界定，它们通常是在后续阶段中不断细化的。

因此，迭代过程这种模式使适应需求的变化会更容易些。

八.统一软件开发过程 RUP 的十大要素

1. 开发前景

2. 达成计划
3. 标识和减小风险
4. 分配和跟踪任务
5. 检查商业理由
6. 设计组件构架
7. 对产品进行增量式的构建和测试
8. 验证和评价结果
9. 管理和控制变化
10. 提供用户支持

让我们逐一的审视这些要素，看一看它们什么地方适合 R U P，找出它们能够成为十大要素的理由。

1. 开发一个前景

有一个清晰的前景是开发一个满足涉众真正需求的产品的关键。前景抓住了 R U P 需求流程的要点：分析问题，理解涉众需求，定义系统，当需求变化时管理需求。前景给更详细的技术需求提供了一个高层的、有时候是合同式的基础。正像这个术语隐含的那样，它是软件项目的一个清晰的、通常是高层的视图，能被??高层的需求和设计约束，让前景的读者能理解将要开发的系统。它还提供了项目审批流程的输入，因此就与商业理由密切相关。最后，

由于前景构成了“项目是什么？”和“为什么要进行这个项目？”，所以可以把前景作为验证将来决策的方式之一。对前景的陈述应该能回答以下问题，需要的话这些问题还可以分成更小、更详细的问题：？关键术语是什么？（词汇表）？我们尝试解决的问题是什么？（问题陈述）？涉众是谁？用户是谁？他们各自的需求是什么？？产品的特性是什么？？功能性需求是什么？（U s e C a s e s）？非功能性需求是什么？？设计约束是什么？

2. 达成计划

“产品的质量只会和产品的计划一样好。” (2) 在 R U P 中，软件开发计划（S D P）综合了管理项目所需的各种信息，也许会包括一些在先启阶段开发的单独的内容。SDP 必须整个项目中被维护和更新。S D P 定义了项目时间表（包括项目计划和迭代计划）和资源需求（资源和工具），可以根据项目进度表来跟踪项目进展。同时也指导了其他过程内容（原文：process components）的计划：项目组织、需求管理计划、配置管理计划、问题解决计划、QA 计划、测试计划、评估计划以及产品验收计划。

在较简单的项目中，对这些计划的陈述可能只有一两句话。比如，配置管理计划可以简单的这样陈述：每天结束时，项目目录的内容将会被压缩成 ZIP 包，拷贝到一个 ZIP 磁盘中，加上日期和版本标签，放到中央档案柜中。软件开发计划的格式远远没有计划活动本身以及驱动这些活动的思想重要。正如 Dwight D.Eisenhower 所说：“plan 什么也不是，planning 才是一切。” “达成计划”——和列表中第 3、4、5、8 条一起——抓住了 RUP 中项目管理流程的要点。项目管理流程包括以下活动：构思项目、评估项目规模 and 风险、监测与控制项目、计划和评估每个迭代和阶段。

3. 标识和减小风险

RUP 的要点之一是在项目早期就标识并处理最大的风险。项目组标识的每一个风险都应该有一个相应的缓解或解决计划。风险列表应该既作为项目活动的计划工具, 又作为确定迭代的基础。

4. 分配和跟踪任务

有一点在任何项目中都是重要的, 即连续的分析来源于正在进行的活动和进化的产品的客观数据。在 RUP 中, 定期的项目状态评估提供了讲述、交流和解决管理问题、技术问题以及项目风险的机制。团队一旦发现了这些障碍物(篱笆), 他们就把所有这些问题都指定一个负责人, 并指定解决日期。进度应该定期跟踪, 如有必要, 更新应该被发布。(原文: updates should be issued as necessary。) 这些项目“快照”突出了需要引起管理注意的问题。随着时间的变化/虽然周期可能会变化(原文: While the period may vary。), 定期的评估使经理能捕获项目的历史, 并且消除任何限制进度的障碍或瓶颈。

5. 检查商业理由

商业理由从商业的角度提供了必要的信息, 以决定一个项目是否值得投资。商业理由还可以帮助开发一个实现项目前景所需的经济计划。它提供了进行项目的理由, 并建立经济约束。当项目继续时, 分析人员用商业理由来正确的估算投资回报率(ROI, 即 return on investment)。商业理由应该给项目创建一个简短但是引人注目的理由, 而不是深入研究问题的细节, 以使所有项目成员容易理解和记住它。在关键里程碑处, 经理应该回顾商业理由, 计算实际的花费、预计的回报, 决定项目是否继续进行。

6. 设计组件构架

在 RUP 中，件系统的构架是指一个系统关键部件的组织或结构，部件之间通过接口交互，而部件是由一些更小的部件和接口组成的。即主要的部分是什么？他们又是怎样结合在一起的？ RUP 提供了一种设计、开发、验证构架的很系统的方法。在分析和设计流程中包括以下步骤：定义候选构架、精化构架、分析行为（用例分析）、设计组件。要陈述和讨论软件构架，你必须先创建一个构架表示方式，以便描述构架的重要方面。在 RUP 中，构架表示由软件构架文档捕获，它给构架提供了多个视图。每个视图都描述了某一组涉众所关心的正在进行的系统的某个方面。涉众有最终用户、设计人员、经理、系统工程师、系统管理员，等等。这个文档使系统构架师和其他项目组成员能与构架相关的重大决策进行有效的交流。

7. 对产品进行增量式的构建和测试

在 RUP 中实现和测试流程的要点是在整个项目生命周期中增量的编码、构建、测试系统组件，在先启之后每个迭代结束时生成可执行版本。在精化阶段后期，已经有了一个可用于评估的构架原型；如有必要，它可以包括一个用户界面原型。然后，在构建阶段的每次迭代中，组件不断的被集成到可执行、经过测试的版本中，不断地向最终产品进化。动态及时的配置管理和复审活动也是这个基本过程元素（原文：essential process element）的关键。

8. 验证和评价结果

顾名思义，RUP 的迭代评估捕获了迭代的结果。评估决定了迭代满足评价标准的程度，还包括学到的教训和实施的过程改进。根据项目的规模和风险以及迭代的特点，评估可以是对演示及其结果的一条简单的纪录，也可能是一个完整的、正式的测试复审记录。这儿的关键是既关注过程问题又关注产品问题。越早发现问题，就越没有问题。（原文：The sooner you fall behind, the more time you will have to catch up.）

9. 管理和控制变化

RUP 的配置和变更管理流程的要点是当变化发生时管理和控制项目的规模，并且贯穿整个生命周期。其目的是考虑所有的涉众需求，尽可能的满足，同时仍能及时的交付合格的产品。用户拿到产品的第一个原型后（往往在这之前就会要求变更），他们会要求变更。重要的是，变更的提出和管理过程始终保持一致。在 RUP 中，变更请求通常用于记录和跟踪缺陷和增强功能的要求，或者对产品提出的任何其他类型的变更请求。变更请求提供了相应的手段来评估一个变更的潜在影响，同时记录就这些变更所作出的决策。他们也帮助确保所有的项目组成员都能理解变更的潜在影响。

10. 提供用户支持

在 RUP 中，部署流程的要点是包装和交付产品，同时交付有助于最终用户学习、使用和维护产品的任何必要的材料。项目组至少要给用户提供一个用户指南（也许是通过联机帮助的方式提供），可能还有一个安装指南和版本发布说明。根据产品的复杂度，用户也许还需要相应的培训材料。最后，通过一个材料清单（BOM 表，即 Bill of Materials）清楚地记录应该和产品一起交付哪些材料。关于需求 有人看了我的要素清单后，可能会非常不同意我的选择。例如，他会问，需求在哪儿呢？他们不重要吗？我会告诉他我为什么没有把它们包括进来。有时，我会问一个项目组（特别是内部项目的项目组）：“你们的需求是什么？”，而得到的回答却是：“我们的确没有什么需求。” 刚开始我对此非常惊讶（我有军方的宇航开发背景）。他们怎么会没有需求呢？当我进一步询问时，我发现，对他们来说，需求意味着一套外部提出的强制性的陈述，要求他们必须怎么样，否则项目验收就不能通过。但是他们的确没有得到这样的陈述。尤其是当项目组陷入了边研究边开发的境地时，产品需求从头到尾都在演化。因此，我接着问他们另外一个问题：“好的，那么你们的产品的前景是

什么呢？” 。这时他们的眼睛亮了起来。然后，我们非常顺利的就第一个要素（“开发一个前景”）中列出的问题进行了沟通，需求也自然而然的流动着（原文：and the requirements just flow naturally.）。也许只有对于按照有明确需求的合同工作的项目组，在要素列表中加入“满足需求”才是有用的。请记住，我的清单仅仅意味着进行进一步讨论的一个起点。

九.总结

RUP 具有很多长处：提高了团队生产力，在迭代的开发过程、需求管理、基于组件的体系结构、可视化软件建模、验证软件质量及控制软件变更等方面，针对所有关键的开发活动为每个开发成员提供了必要的准则、模板和工具指导，并确保全体成员共享相同的知识基础。它建立了简洁和清晰的过程结构，为开发过程提供较大的通用性。但同时它也存在一些不足：RUP 只是一个开发过程，并没有涵盖软件过程的全部内容，例如它缺少关于软件运行和支持等方面的内容；此外，它没有支持多项目的开发结构，这在一定程度上降低了在开发组织内大范围实现重用的可能性。可以说 RUP 是一个非常好的开端，但并不完美，在实际的应用中可以根据需要对其进行改进并可以用 OPEN 和 OOSP 等其他软件过程的相关内容对 RUP 进行补充和完善。